METHODOLOGY REPORT: MBTI PERSONALITY CLASSIFICATION FROM TEXT

1.0 INTRODUCTION

This report details the systematic methodology employed to develop and evaluate a machine learning model capable of classifying Myers-Briggs Type Indicator (MBTI) personality types from user-generated text. The primary objective of this project is to create a robust classifier that can accurately infer personality traits based on written content. The core methodological stages covered herein include comprehensive data preprocessing to clean and structure raw text, advanced feature engineering using a dual-level Term Frequency-Inverse Document Frequency (TF-IDF) strategy, a comparative evaluation of three distinct classification models, and an in-depth analysis of the final model's performance and interpretability. This document will begin by outlining the initial data acquisition and exploratory analysis, which formed the foundation for all subsequent modeling decisions.

2.0 DATA ACQUISITION AND EXPLORATORY ANALYSIS

Understanding the fundamental characteristics of the source data is a critical prerequisite for any successful machine learning project. This exploratory phase provides essential insights into the dataset's structure, quality, and statistical distribution. The findings from this analysis directly inform the design of the data preprocessing pipeline, the approach to feature engineering, and the strategies for mitigating potential biases, such as class imbalance, during model training.

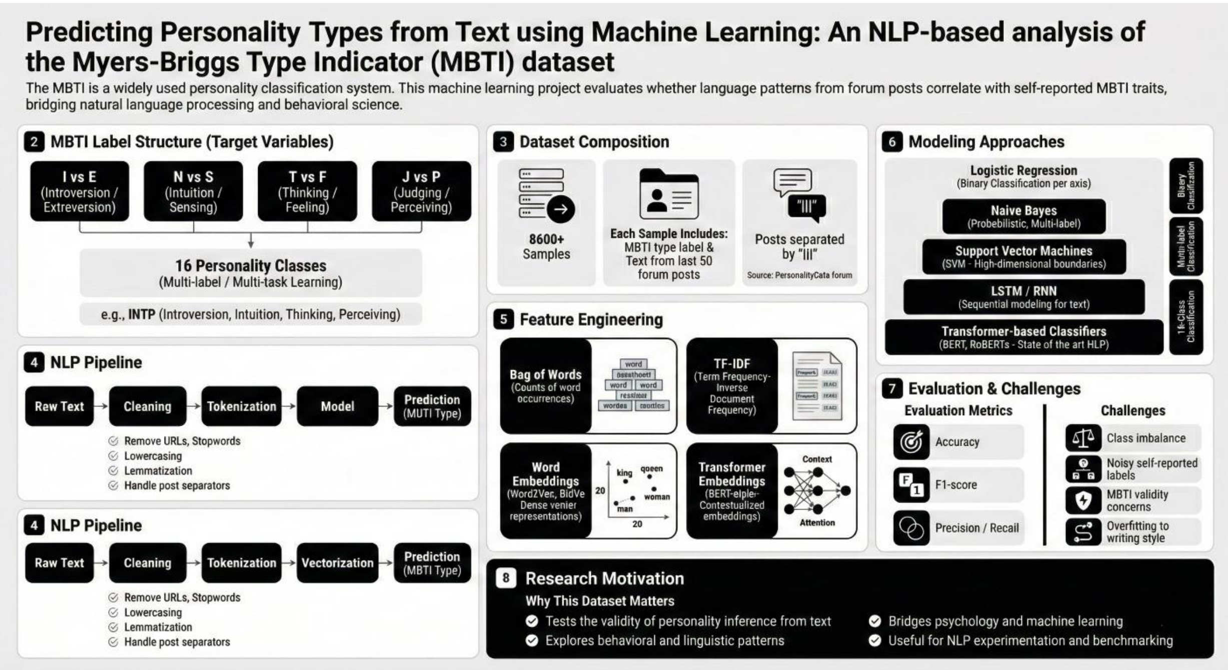

Figure 1: Dataset Description

2.1 DATASET DESCRIPTION

The project utilizes the mbti_1.csv dataset as its primary source of training data. This dataset is structured into 8,675 individual records, with each record representing a single user. The data is organized into two primary columns:

- 'type': The target variable, containing the 16-class MBTI personality label for the user (e.g., 'INFP', 'ENTJ').
- 'posts': The input feature, consisting of a single text field where multiple posts from a user have been concatenated.

2.2 INITIAL DATA QUALITY AND DISTRIBUTION

Initial data quality checks were performed to identify potential issues that could impact model performance. These checks confirmed that the dataset is clean, with **no missing values** in either column and **no duplicate posts** across the 8,675 records.

An analysis of the distribution of the 16 MBTI types revealed a significant class imbalance, with some personality types being far more prevalent than others. The five most frequent types in the dataset are:

- **INFP:** 1832 records
- **INFJ:** 1470 records
- **INTP:** 1304 records
- **INTJ:** 1091 records
- **ENTP:** 685 records

When broken down into the four binary axes that comprise the MBTI framework, the imbalance is also evident:

- **Introversion (I) / Extraversion (E):** I: 6676, E: 1999
- **Intuition (N) / Sensing (S):** N: 7478, S: 1197
- **Feeling (F) / Thinking (T):** F: 4694, T: 3981
- **Judging (J) / Perceiving (P):** J: 5241, P: 3434

The insights gained from this exploratory analysis, particularly the confirmation of class imbalance, underscore the necessity of the rigorous preprocessing and model configuration steps detailed in the following sections.

3.0 DATA PREPROCESSING AND PREPARATION PIPELINE

Raw user-generated text is inherently unstructured and contains significant noise, such as URLs, special characters, and stylistic variations. To prepare this data for a machine learning model, a multi-step preprocessing pipeline was implemented. This pipeline transforms the raw text into a clean, standardized, and tokenized format, making it suitable for numerical feature extraction.

3.1 TEXT CLEANING AND NORMALIZATION

A sequence of text cleaning operations was applied to each user's concatenated posts to standardize the corpus. These operations were executed in the following order:

1. **URL Removal:** All URLs were identified using regular expressions and removed from the text to prevent them from being treated as meaningful features.
2. **Standardization:** All text was converted to lowercase to ensure uniformity and prevent the model from treating the same word with different capitalization (e.g., 'Think' and 'think') as distinct features.
3. **Special Character Removal:** All non-alphanumeric characters, including punctuation and symbols, were removed to isolate the core textual content.

4. **Tokenization:** The cleaned text was segmented into individual words or "tokens," creating a structured list of terms for each user.
5. **Stopword Removal:** Common English stopwords (e.g., 'the', 'is', 'a', 'and'), which provide little semantic value for classification, were removed from the token lists.
6. **Lemmatization:** Each token was reduced to its base or dictionary root form (lemma). For example, words like 'running,' 'runs,' and 'ran' were all converted to 'run,' consolidating their semantic meaning.

3.2 TARGET VARIABLE TRANSFORMATION

Instead of approaching this as a single, complex 16-class classification problem, the task was reframed as four independent binary classification problems. The 'type' column was deconstructed into four new binary target columns, each representing one of the MBTI axes:

- 'IE' (Introversion/Extraversion)
- 'SN' (Intuition/Sensing)
- 'TF' (Thinking/Feeling)
- 'JP' (Judging/Perceiving)

Each of these columns was then numerically encoded, with one personality trait represented by 1 and its opposite by 0.

3.3 DATASET SPLITTING

The fully preprocessed dataset was divided into three distinct subsets to ensure a robust and unbiased model evaluation:

- **Training Set (70%):** 6072 samples used to train the machine learning models.
- **Validation Set (15%):** 1301 samples used to tune hyperparameters and perform intermediate model checks.
- **Test Set (15%):** The remaining samples held out for the final, definitive performance evaluation of the chosen models.

To preserve the class distribution observed in the original dataset, the split was performed using **stratification** based on the 'IE' axis. This ensures that the proportion of Introverts to Extraverts is consistent across the training, validation, and test sets.

With the text cleaned and the data structured into training and testing sets, the next step was to convert the processed text into a numerical format that machine learning algorithms can interpret.

4.0 FEATURE ENGINEERING WITH TF-IDF

To enable machine learning algorithms to process the cleaned text, a feature extraction technique was required to convert the textual data into a numerical representation. Term Frequency-Inverse Document Frequency (TF-IDF) was selected as the primary method for this task. TF-IDF is highly effective for text classification as it quantifies the importance of a word not just by its frequency within a single document, but also by its rarity across the entire corpus, giving more weight to distinctive and informative terms.

4.1 DUAL-LEVEL VECTORIZATION STRATEGY

A two-pronged vectorization strategy was implemented to capture a comprehensive range of textual patterns, combining both word-level and character-level features:

- **Word-Level Vectorization:** A TfidfVectorizer was configured to analyze the text at the word level. It was set to extract the **top 20,000 most frequent words** across the entire training corpus, creating a feature set based on the TF-IDF scores of these individual terms.
- **Character-Level Vectorization:** A second TfidfVectorizer was used to create features from sequences of characters (n-grams). This vectorizer was configured to generate features from character n-grams ranging from **3 to 5 characters** in length and was limited to the **top 30,000 most frequent n-grams**. This approach helps capture sub-word patterns, misspellings, and stylistic markers that word-level analysis might miss.

4.2 FEATURE ANALYSIS AND COMBINATION

The feature sets generated from the word-level and character-level vectorizers were combined into a single, high-dimensional sparse matrix. This unified matrix served as the final input for the machine learning models.

An analysis of the Inverse Document Frequency (IDF) scores from the word-level vectorizer illustrates how the technique distinguishes between common and rare words. Low IDF scores indicate common words found in many documents, while high IDF scores point to rare words that may be highly indicative of a specific topic or group.

Low IDF (Common Words)	High IDF (Rare Words)
like	regard hi
think	miranda infj
people	chung daydream
know	xoxo miranda
time	naomi chung

This engineered feature set, which captures both semantic and stylistic information, was then used to train and evaluate the candidate classification models.

5.0 MODEL TRAINING AND SELECTION

The model selection strategy was designed to identify the most effective classification algorithm for this multi-output prediction task. Three distinct models, each with different architectural strengths, were chosen to evaluate which approach would yield the best performance on the engineered TF-IDF feature set.

5.1 CANDIDATE MODELS

The following three algorithms were selected for comparative evaluation:

- **Logistic Regression:** A linear model chosen as a strong, interpretable, and computationally efficient baseline. It is often very effective for text classification tasks.
- **Linear Support Vector Machine (SVM):** A powerful and robust linear model, widely recognized for its high performance in text classification and its ability to handle high-dimensional sparse data effectively.
- **Random Forest:** A complex, non-linear ensemble model composed of multiple decision trees. It was selected to determine if capturing non-linear relationships between features would provide a performance advantage.

5.2 TRAINING CONFIGURATION

To handle the task of predicting four independent binary outcomes (I/E, S/N, T/F, J/P) simultaneously, a `MultiOutputClassifier` wrapper was utilized. This wrapper trains a separate classifier for each of the four target axes, allowing each model architecture to be applied to the multi-label problem.

Crucially, to address the class imbalance identified during the exploratory data analysis, the `class_weight="balanced"` parameter was set for all three models. This configuration automatically adjusts the model's learning process to give more weight to the minority class in each binary axis, preventing the model from becoming biased towards the more frequent class.

With the models trained, the next stage was to conduct a rigorous and multi-faceted evaluation of their performance.

6.0 PERFORMANCE EVALUATION AND RESULTS

A comprehensive evaluation framework was established to assess and compare the performance of the three trained models. The primary evaluation metrics used were **Accuracy** and **Macro F1-Score**. While accuracy measures the overall proportion of correct predictions, the Macro F1-Score is particularly critical for this task. It calculates the F1-score for each class independently and then takes the average, ensuring that the model's performance on infrequent classes is weighted equally to its performance on frequent classes. This makes it a more reliable metric for imbalanced datasets.

6.1 PER-AXIS PERFORMANCE ON TEST SET

The models were first evaluated on their ability to predict each of the four MBTI axes independently. The results on the unseen test set clearly show that the linear models significantly outperformed the ensemble model.

Model	I/E F1	S/N F1	T/F F1	J/P F1	Per-Label Acc	Macro-F1
Logistic Regression	0.80	0.82	0.80	0.77	0.8567	**0.8643**
Linear SVM	0.79	0.79	0.77	0.71	0.8452	**0.8519**
Random Forest	0.46	0.48	0.71	0.64	0.7897	0.7633

While the linear models consistently outperformed Random Forest, their strengths varied. Both excelled on the highly imbalanced S/N axis (F1 > 0.78), but Logistic Regression showed a notable advantage on the T/F and J/P axes, suggesting its probabilistic nature was better suited for those particular linguistic separations. Random Forest struggled significantly with the imbalanced I/E and S/N dimensions, as reflected by its low F1-scores.

6.2 OVERALL 16-CLASS PERFORMANCE ON TEST SET

The models were also evaluated on the much more difficult task of predicting the exact 16-type personality combination correctly. As expected, performance on this task was substantially lower, highlighting the combinatorial challenge of getting all four axes correct simultaneously. This exponential increase in difficulty is expected; for a model that is 85% accurate on each of four independent binary tasks, the theoretical probability of correctly predicting all four simultaneously is $(0.85)^4$, which equals approximately 52%.

Model	Exact Accuracy	Macro-F1
Logistic Regression	0.5606	0.3922
Linear SVM	0.5560	0.2975
Random Forest	0.4101	0.1292

The Logistic Regression model achieved the highest Macro F1-Score (0.392) in this challenging 16-class scenario. This significant drop from the per-axis results underscores that while the models are very effective at predicting individual traits, correctly identifying the complete four-letter type is a much harder problem.

6.3 QUALITATIVE ANALYSIS VIA VISUALIZATIONS

To gain deeper insights into model behavior, qualitative analyses were performed using visualizations.

- **Confusion Matrices:** Confusion matrices were generated for each of the four axes. These visualizations provide a detailed breakdown of model performance by showing the exact counts of true positives, true negatives, false positives, and false negatives for each class. This allows for a granular diagnosis of where a model is succeeding or making errors (e.g., misclassifying 'E' as 'I').

- **Calibration Curves:** Calibration curves were generated for the SVM model to assess the reliability of its predicted probabilities. These plots help determine if the confidence scores produced by the model are well-calibrated, for instance, if a prediction with an 80% confidence score is actually correct 80% of the time. This is a crucial diagnostic for applications where understanding the model's confidence is important.

7.0 ADVANCED MODEL ANALYSIS

Beyond standard performance metrics, additional analyses were conducted to test the model's robustness under different conditions and to interpret which features were most influential in its decision-making process.

7.1 ROBUSTNESS TO INPUT DATA VOLUME

A key question for practical application is how the model performs when less text is available. A robustness test was conducted by evaluating the model's performance on a progressively smaller number of posts per user: 50, 10, and just 1.

The Macro-F1 scores for each input limit were as follows:

- **50 posts:** 0.7957
- **10 posts:** 0.7778
- **1 post:** 0.6063

This analysis reveals that model performance degrades gracefully as the volume of input text decreases. The model remains reasonably effective even with as few as 10 posts but experiences a significant drop in performance when only a single post is available, which is an expected outcome given the reduced information content.

7.2 FEATURE IMPORTANCE INTERPRETATION

To understand *why* the models make certain predictions, the most influential features (words) for the Logistic Regression model were extracted for each personality axis. This provides valuable insight into the linguistic patterns associated with each trait.

AXIS I/E (INTROVERSION/EXTRAVERSION):

- **Class 1 (+) Features (Introversion):** `intps, intjs, infjs, enfp, entp, intj, infp, infj, intp`
- **Class 0 (-) Features (Extraversion):** `music, isfp, isfj, istp, istps, enfps, enfj, estp, ne, fun`

AXIS S/N (INTUITION/SENSING):

- **Class 1 (+) Features (Intuition):** `intjs, infjs, enfj, infp, entp, intj, intp`
- **Class 0 (-) Features (Sensing):** `isfp, isfj, istp, esfp, istj, isfps, estp, rant, esfjs`

AXIS T/F (THINKING/FEELING):

- **Class 1 (+) Features (Thinking):** `intp, entj, istp, intj, entps, entp`
- **Class 0 (-) Features (Feeling):** `infp, infj, enfp, feel, love, infps, infjs, isfp, feeling`

AXIS J/P (JUDGING/PERCEIVING):

- **Class 1 (+) Features (Judging):** `plan, entj, enfj, isfj, intjs, ni, infjs, intj, infj`
- **Class 0 (-) Features (Perceiving):** `infp, entp, intp, enfp, infps, istp, intps, ne, entps, isfp`

These analyses confirm the model's robustness and provide a degree of interpretability, building confidence in its predictive capabilities.

8.0 CONCLUSION AND FINAL MODEL SELECTION

This report has detailed a comprehensive workflow for classifying MBTI personality types from text, encompassing data cleaning, a dual-level TF-IDF feature engineering strategy, and a comparative evaluation of Logistic Regression, Linear SVM, and Random Forest models. The evaluation results consistently demonstrated that the linear models, Logistic Regression and Linear SVM, significantly outperformed the more complex Random Forest model, particularly in the critical per-axis classification task where they achieved superior Macro F1-Scores. The superior performance of Logistic Regression and Linear SVM highlights their effectiveness on high-dimensional, sparse text data, where complex non-linear models like Random Forest can be more susceptible to overfitting.

Based on this comprehensive performance analysis, the **Linear SVM model** or the **Logistic Regression model** are the top candidates for deployment. The project documentation indicates a final decision was made to proceed with the SVM model for integration into a user-facing application.

The final step of this project was to serialize and save the trained TF-IDF vectorizer and the selected SVM model. This ensures that the entire preprocessing and prediction pipeline can be reliably loaded and used for making inferences on new, unseen text data in a production environment.